C++ STL Lecture Notes: Functions & Examples

Lecture Handout

1 Part 1: Sequence Containers

1.1 1. std::vector (Dynamic Array)

The default container. Use this 90% of the time.

- **Capacity**:
  - `size()`: Returns number of elements.
  - `empty()`: Returns true if size is 0.
  - `reserve(n)`: Allocates memory upfront (optimization).
- **Modifiers**:
  - `push_back(val)`: Adds element to end. $O(1)$ amortized.
  - `pop_back()`: Removes last element. $O(1)$.
  - `clear()`: Removes all elements.
  - `erase(iterator)`: Removes element at position. $O(N)$.
- **Access**:
  - `v[i]`: Direct access (no bounds checking).
  - `front()`, `back()`: Access first/last element.

Code Example:

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);

    cout << "Size: " << v.size() << endl;
    cout << "Second element: " << v[1] << endl;

    v.pop_back(); // Removes 30
    for(int x : v) cout << x << " ";
    return 0;
}
```

Expected Output:

```
Size: 3
Second element: 20
10 20
```

1.2 2. std::string (Character Sequence)

Treat strings like vectors with special text manipulation functions.

- `length()` / `size()`: Returns length.
- `substr(pos, len)`: Returns a sub-string.
- `find(str)`: Returns index of first occurrence or `string::npos`.
- `getline(cin, str)`: Reads a full line including spaces.

Code Example:

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s = "Hello World";
    cout << "Length: " << s.length() << endl;
    cout << "Substr: " << s.substr(0, 5) << endl;

    size_t found = s.find("World");
    if (found != string::npos)
        cout << "'World' found at index: " << found;
    return 0;
}
```

Expected Output:

```
Length: 11
Substr: Hello
'World' found at index: 6
```

1.3 3. std::deque (Double Ended Queue)

Allows fast insertion at both the front and back.

- `push_front(val)`: Insert at beginning. $O(1)$.
- `pop_front()`: Remove from beginning. $O(1)$.
- (Plus all vector functions like `push_back`).

Code Example:

```cpp
#include <iostream>
#include <deque>
using namespace std;

int main() {
    deque<int> d;
    d.push_back(10);
    d.push_front(5);
    d.push_back(20);

    cout << "Front: " << d.front() << ", Back: " << d.back();
    return 0;
}
```

Expected Output:

```
Front: 5, Back: 20
```

2 Part 2: Container Adapters

2.1 1. std::stack (LIFO - Last In, First Out)

- `push(val)`: Adds to top.
- `pop()`: Removes top.
- `top()`: Returns the top element.

Code Example:

```cpp
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> s;
    s.push(100);
    s.push(200); // 200 is on top

    cout << "Top: " << s.top() << endl;
    s.pop(); // Removes 200
    cout << "New Top: " << s.top();
    return 0;
}
```

Expected Output:

```
Top: 200
New Top: 100
```

2.2 2. std::queue (FIFO - First In, First Out)

- `push(val)`: Adds to the back.
- `pop()`: Removes from the front.
- `front()`: Returns the front element.

Code Example:

```cpp
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;
    q.push(10);
    q.push(20);

    cout << "Front: " << q.front() << endl;
    q.pop(); // Removes 10
    cout << "New Front: " << q.front();
    return 0;
}
```

Expected Output:

```
Front: 10
New Front: 20
```

2.3 3. std::priority_queue (Heap)

Defaults to Max-Heap (largest element on top).

- `push(val)`: Insert element. $O(\log N)$.
- `pop()`: Remove top element. $O(\log N)$.
- `top()`: Access top element. $O(1)$.

Code Example:

```cpp
#include <iostream>
#include <queue>
using namespace std;

int main() {
    priority_queue<int> pq;
    pq.push(10);
    pq.push(50);
    pq.push(20);

    // Largest element is always on top
    cout << "Top (Max): " << pq.top() << endl;
    pq.pop(); // Removes 50
    cout << "Next Max: " << pq.top();
    return 0;
}
```

Expected Output:

```
Top (Max): 50
Next Max: 20
```

3 Part 3: Associative Containers

3.1 1. std::set (Balanced BST)

Stores unique elements in sorted order.

- `insert(val)`: Adds element. $O(\log N)$.
- `erase(val)`: Removes element. $O(\log N)$.
- `count(val)`: Returns 1 if exists, 0 otherwise.

Code Example:

```cpp
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s;
    s.insert(5);
    s.insert(1);
    s.insert(5); // Duplicate ignored
    s.insert(3);

    // Iterates in sorted order automatically
    for(auto x : s) cout << x << " ";
    return 0;
}
```

Expected Output:

```
1 3 5
```

3.2 2. std::map (Key-Value Pairs)

Keys are unique and sorted.

- `insert({key, val})`: Inserts pair.
- `m[key]`: Access value. **Warning:** Creates key if it doesn't exist.
- `count(key)`: Checks if key exists.

Code Example:

```cpp
#include <iostream>
#include <map>
#include <string>
using namespace std;

int main() {
    map<string, int> m;
    m["apple"] = 5;
    m["banana"] = 10;
    m["apple"] = 8; // Overwrites value

    if(m.count("banana")) {
        cout << "Banana cost: " << m["banana"] << endl;
    }
    cout << "Apple cost: " << m["apple"];
    return 0;
}
```

Expected Output:

```
Banana cost: 10
Apple cost: 8
```

4 Part 4: Algorithms

4.1 1. Sorting & Reversing

- `sort(begin, end)`: Introsort. $O(N \log N)$.
- `reverse(begin, end)`: Reverses range.

Code Example:

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {4, 1, 3, 2};

    sort(v.begin(), v.end()); // Sort ascending
    cout << "Sorted: ";
    for(int x : v) cout << x << " ";

    reverse(v.begin(), v.end());
    cout << "\nReversed: ";
    for(int x : v) cout << x << " ";
    return 0;
}
```

Expected Output:

```
Sorted: 1 2 3 4
Reversed: 4 3 2 1
```

4.2 2. Binary Search

Requires the container to be sorted first.

- `binary_search(begin, end, val)`: Returns true/false.
- `lower_bound(begin, end, val)`: Iterator to first element $\geq$ val.

Code Example:

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30, 40, 50}; // Must be sorted

    // lower_bound finds first element >= value
    auto it = lower_bound(v.begin(), v.end(), 35);

    cout << "Found index: " << (it - v.begin()) << endl;
    cout << "Value found: " << *it;
    return 0;
}
```

Expected Output:

```
Found index: 3
Value found: 40
```

4.3 3. Min/Max & Math

- `*max_element(begin, end)`: Finds largest value.
- `accumulate(begin, end, 0)`: Sums up range (needs `<numeric>`).

Code Example:

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
using namespace std;

int main() {
    vector<int> v = {10, 5, 80, 2};

    int mx = *max_element(v.begin(), v.end());
    int mn = *min_element(v.begin(), v.end());
    int sum = accumulate(v.begin(), v.end(), 0);

    cout << "Max: " << mx << ", Min: " << mn << ", Sum: " << sum;
    return 0;
}
```

Expected Output:

```
Max: 80, Min: 2, Sum: 97
```

5 Part 5: Utilities

5.1 1. Pairs & Range-based Loops

- `make_pair(a, b)`: Creates a pair.
- `for(int &x : v)`: Loop by reference (modifiable).

Code Example:

```cpp
#include <iostream>
#include <vector>
#include <utility>
using namespace std;

int main() {
    // Pairs
    pair<string, int> p = {"Alice", 95};
    cout << "Name: " << p.first << ", Score: " << p.second << endl;

    // Reference Loop
    vector<int> v = {1, 2, 3};
    for(int &x : v) x = x * 2; // Modify values

    cout << "Doubled: ";
    for(int x : v) cout << x << " ";
    return 0;
}
```

Expected Output:

```
Name: Alice, Score: 95
Doubled: 2 4 6
```